

Evolution of Sonar & Sonar Signal Processing Fundamentals

Instructors: Dr. Spencer J. Chang, Dr. Daniel D. Sternlicht

Notebook Description: This notebook covers the basic concepts and exercises for processing sonar signals after they have been digitized with an A/D converter.

Distribution A: Approved for public release. Distribution is unlimited.

Content is largely based on standard reference materials.

```
In [1]: ##### Python Imports across the whole notebook #####
import os, struct
import numpy as np
import scipy
import matplotlib.pyplot as plt

# It would be informative to skim the attributes
# for the stave data to know what is getting loaded.
from ReadStaveData import StaveData
from joblib import Parallel, delayed
```

Signal Characteristics

If we want to do analog signal processing, we must use *analog* components, like resistors, capacitors, inductors, transistors, and operational amplifiers in circuits that modify received signals. If we want to do **digital signal processing** as we are in this tutorial, we must first receive the signal through an analog-to-digital converter.

This carries with it a series of implications. One of the biggest implications that affects us is the chosen sampling rate.

What do we mean by sampling rate?

When collecting the amplitude information of incoming signals, we're restricted by our hardware (e.g., how many different amplitude levels we can receive) and *by how often we record the amplitude value of the signal*.

Note: `numpy.sinc` computes the *normalized sinc* function $\frac{\sin(\pi x)}{\pi x}$, so to get a signal up to 15 Hz, $x = 2(15)t$.

```
In [2]: # -----
# 1. Define a time axis (e.g., 0-2 seconds, 1 kHz sampling)
# -----
fs = 20                # Sampling frequency [Hz]
t_end = 2              # Duration [seconds]
t = np.arange(0, t_end, 1/fs)

# -----
# 2. Generate (normalized) sinc function signals
# -----
impulse_f = 20
sig_a = lambda t_val: scipy.signal.hilbert(1.0 * np.sinc(2*impulse_f*(t_val-0.412)))
sig_b = lambda t_val: scipy.signal.hilbert(1.0 * np.sinc(2*impulse_f*(t_val-0.637)))
sig_c = lambda t_val: scipy.signal.hilbert(1.0 * np.sinc(2*impulse_f*(t_val-0.8)))
sig_d = lambda t_val: scipy.signal.hilbert(1.0 * np.sinc(2*impulse_f*(t_val-1)))

# -----
# 3. Plot everything for visual verification
# -----
sig_analysis = sig_a    # Change this to look at (potential) differences in FFT output

plt.figure(figsize=(10, 5))

# Plot each component
plt.subplot(2, 1, 1)
plt.plot(t, sig_a(t).real)
plt.plot(t, sig_b(t).real)
plt.plot(t, sig_c(t).real)
plt.plot(t, sig_d(t).real)
plt.title('Individual Sinusoidal Components')
plt.xlabel('Time [s]')
```

```

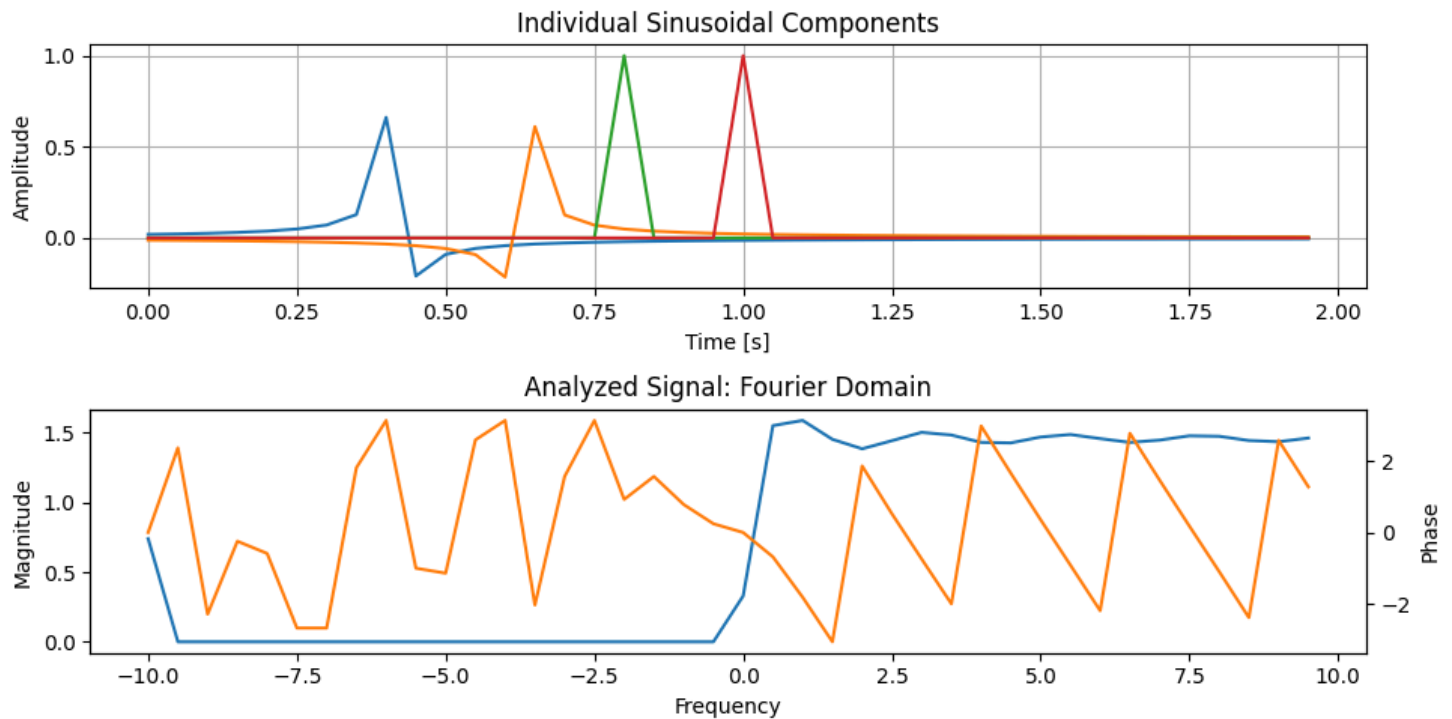
plt.ylabel('Amplitude')
plt.grid(True)

n_fft = len(t)
sig_fft = scipy.fft.fft(sig_analysis(t), n=n_fft)
freq_axis = scipy.fft.fftfreq(n=n_fft, d=1/fs)

plt.subplot(2, 1, 2)
plt.plot(scipy.fft.fftshift(freq_axis),
         scipy.fft.fftshift(np.abs(sig_fft)))
plt.title("Analyzed Signal: Fourier Domain")
plt.xlabel("Frequency")
plt.ylabel("Magnitude")

tw_ax = plt.twinx(plt.gca())
tw_ax.plot(scipy.fft.fftshift(freq_axis),
          scipy.fft.fftshift(np.angle(sig_fft)), c='tab:orange')
tw_ax.set_ylabel("Phase")
plt.tight_layout();

```



Aliasing

```

In [3]: # Example of aliasing when undersampling a signal.
fs = 60 # Sampling frequency [Hz]
t_end = 2 # Duration [seconds]
t = np.arange(0, t_end, 1/fs)

s_freq = 20
c_freq = 50
sine_cos = lambda t_val: np.sin(2*np.pi*s_freq*(t_val-0.412)) + np.cos(2*np.pi*c_freq*t_val)

plt.figure(figsize=(10, 5))

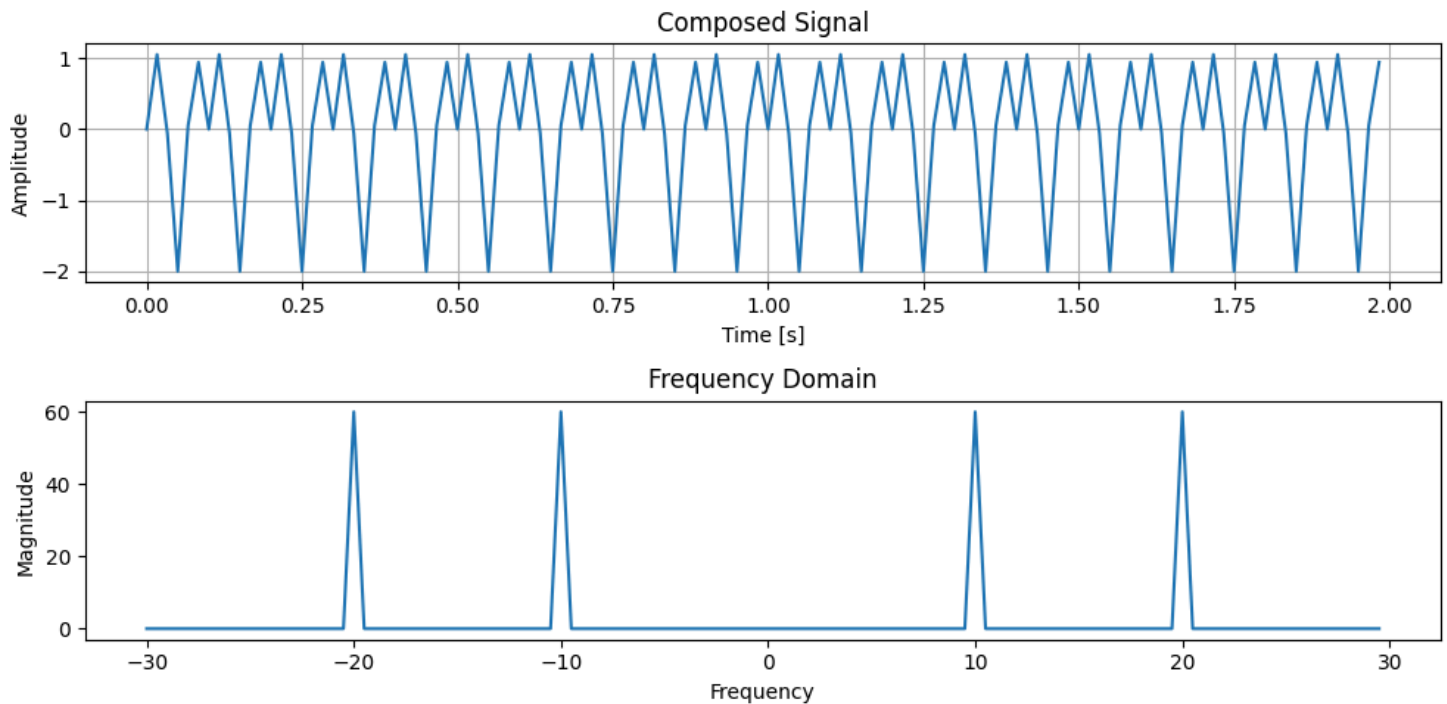
# Plot each component
plt.subplot(2, 1, 1)
plt.plot(t, sine_cos(t))
plt.title('Composed Signal')
plt.xlabel('Time [s]')
plt.ylabel('Amplitude')
plt.grid(True)

n_fft = len(t)
sig_fft = scipy.fft.fft(sine_cos(t), n=n_fft)
freq_axis = scipy.fft.fftfreq(n=n_fft, d=1/fs)

plt.subplot(2, 1, 2)
plt.plot(scipy.fft.fftshift(freq_axis),
         scipy.fft.fftshift(np.abs(sig_fft)))

```

```
plt.title("Frequency Domain")
plt.xlabel("Frequency")
plt.ylabel("Magnitude")
plt.tight_layout();
```



Notice how the 50 Hz sinusoid is show up as approximately 7 Hz instead. When we undersample the input data, aliasing of the signal is often what happens. Essentially, the sinusoid's original 50 Hz frequency becomes "known as" (aliased) a 10 Hz signal.

Passband and Baseband

Other terms that serve similar purposes:

- Carrier frequency (passband)
- Base frequency (baseband)
- Complex envelope (baseband)
- (Carrier) Modulated signal (passband)
- Modulating signal (baseband)
- Intermediate/Physical/Real signal (passband)

Below is a practical example of how to shift frequencies up and down in software.

```
In [4]: # Example: Frequency shift of signal in time-/frequency-domain.
s_freq = 20
freq_axis = scipy.fft.fftfreq(n=len(t), d=1/fs)
sig_data = np.sin(2*np.pi*s_freq*t)
sig_data = scipy.signal.hilbert(sig_data)

## Time-domain frequency-shifting with Euler's equation
shift_down = sig_data * np.exp(-2j * np.pi * 4 * t)
shift_up = sig_data * np.exp(2j * np.pi * 4 * t)
sig_fft = scipy.fft.fft(sig_data)
down_fft = scipy.fft.fft(shift_down)
up_fft = scipy.fft.fft(shift_up)

## Frequency-domain frequency-shifting (uses frequency sampling)
shift_hz = 3
bin_spacing = fs / len(t)
bins_to_shift = int(round(shift_hz / bin_spacing))
down_fft_2 = np.roll(down_fft, -bins_to_shift)

## Time-domain signals
down_time = np.fft.ifft(down_fft)
up_time = np.fft.ifft(up_fft)
down_time_2 = np.fft.ifft(down_fft_2)

plt.figure(figsize=(7, 3))
```

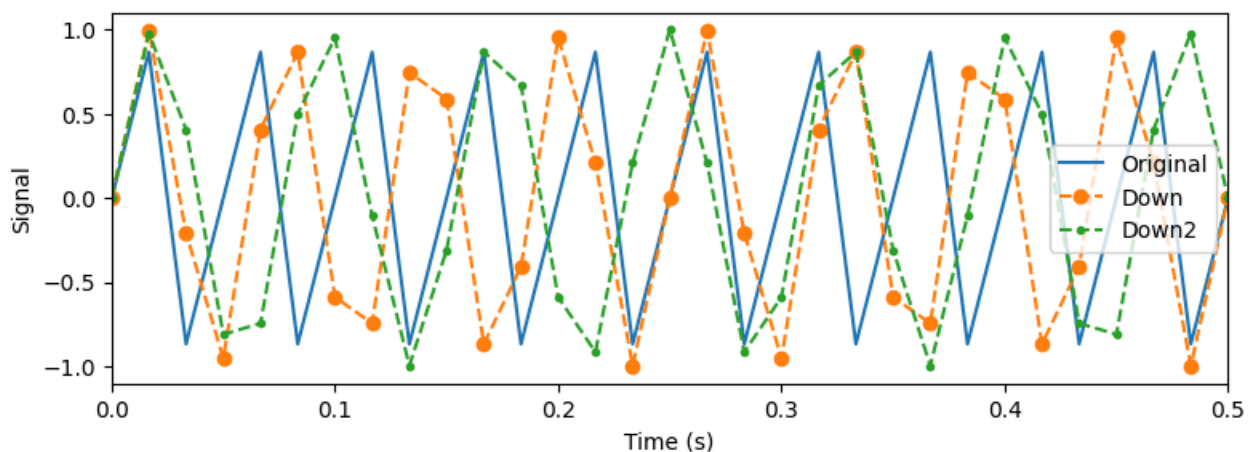
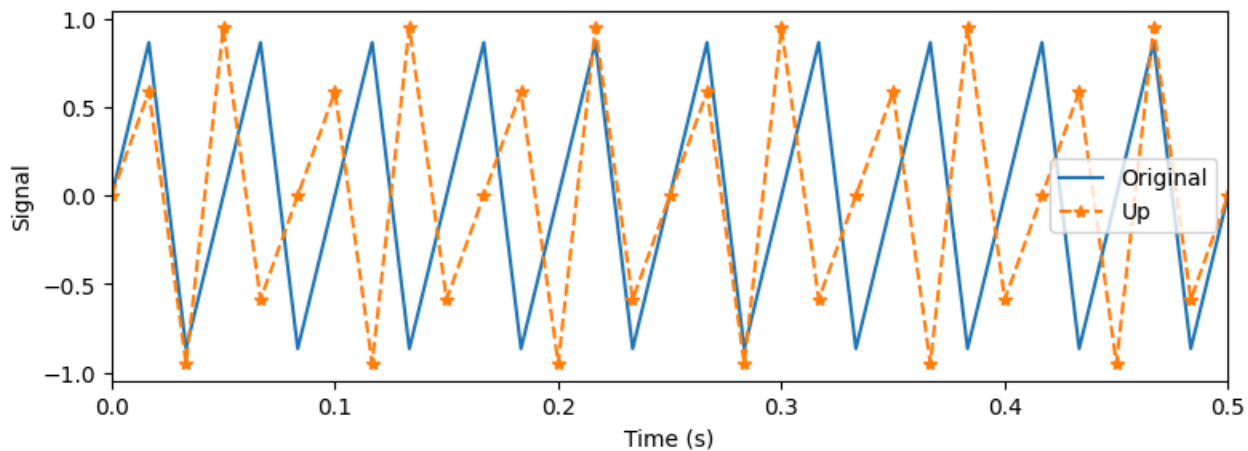
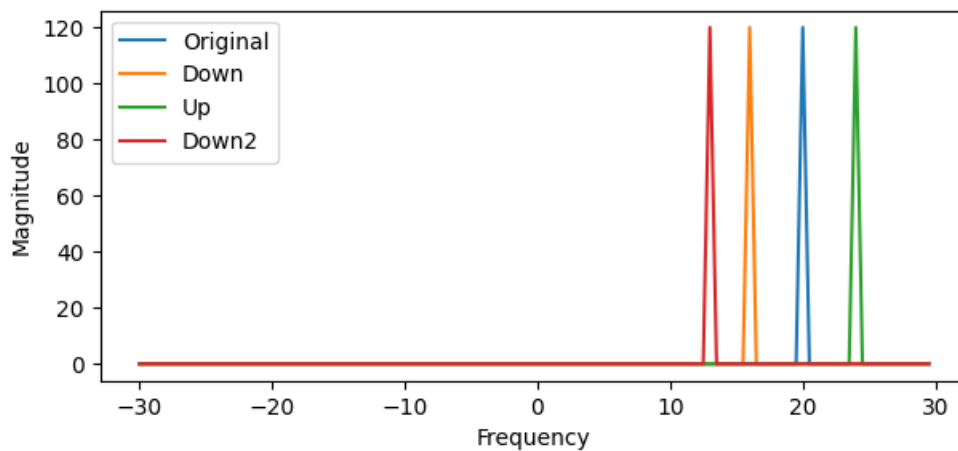
```

plt.plot(scipy.fft.fftfreq(freq_axis), scipy.fft.fftshift(np.abs(sig_fft)), label="Original")
plt.plot(scipy.fft.fftfreq(freq_axis), scipy.fft.fftshift(np.abs(down_fft)), label="Down")
plt.plot(scipy.fft.fftfreq(freq_axis), scipy.fft.fftshift(np.abs(up_fft)), label="Up")
plt.plot(scipy.fft.fftfreq(freq_axis), scipy.fft.fftshift(np.abs(down_fft_2)), label="Down2")
plt.xlabel("Frequency")
plt.ylabel("Magnitude")
plt.legend();

plt.figure(figsize=(9, 3))
plt.plot(t, sig_data.real, label="Original")
plt.plot(t, up_time.real, "*--", label="Up")
plt.xlabel("Time (s)")
plt.ylabel("Signal")
plt.xlim(0, 0.5)
plt.legend();

plt.figure(figsize=(9, 3))
plt.plot(t, sig_data.real, label="Original")
plt.plot(t, down_time.real, "o--", label="Down")
plt.plot(t, down_time_2.real, "-.-", label="Down2")
plt.xlabel("Time (s)")
plt.ylabel("Signal")
plt.xlim(0, 0.5)
plt.legend();

```



Phase Information

Example: Determining time-of-arrival for a received impulse signal. Notice that the magnitude in the frequency domain is always the same, but the phase information changes.

```
In [5]: fs = 60
t_end = 2 # Duration [seconds]
t = np.arange(0, t_end, 1/fs)

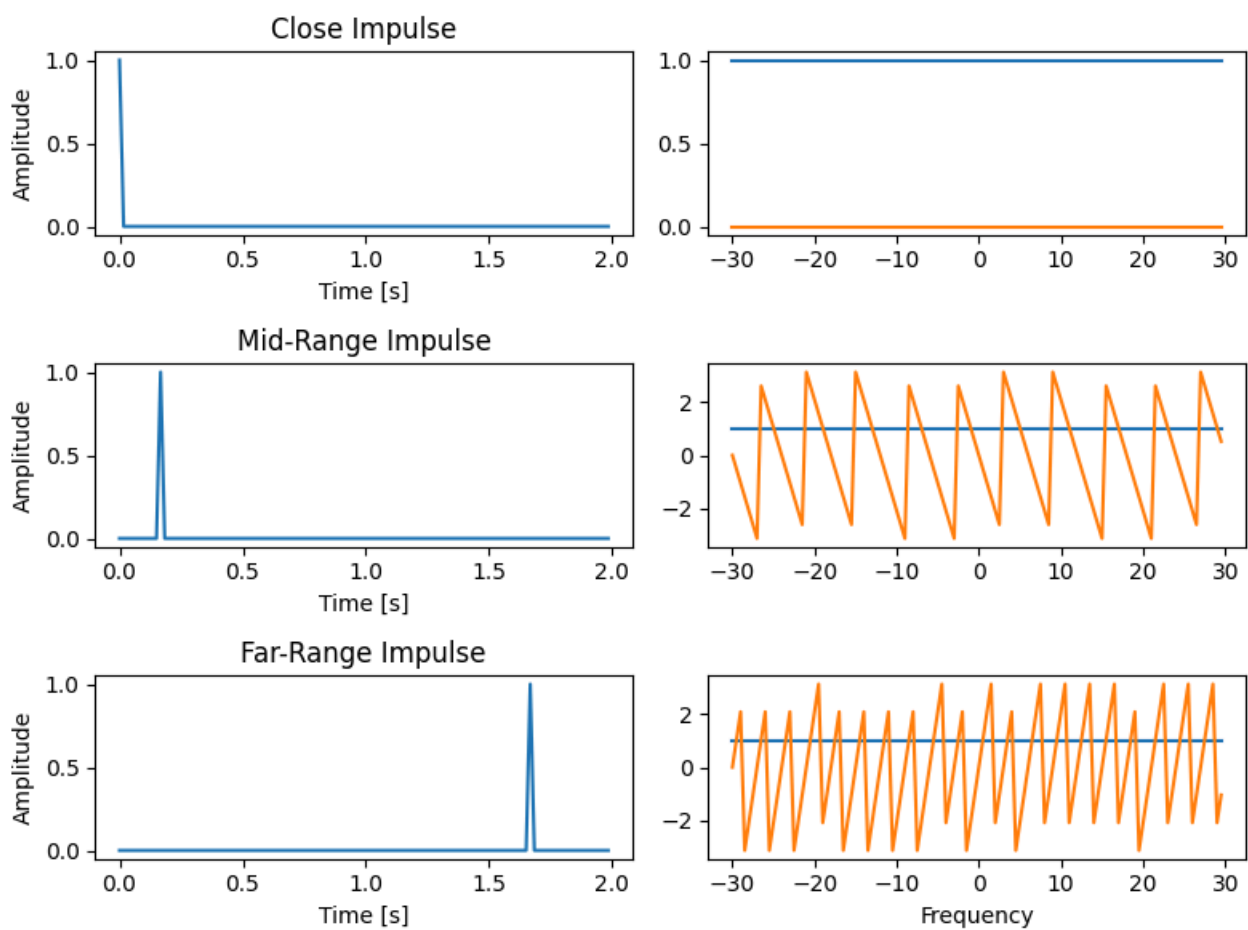
sig_close = np.pad([1], (0, len(t)-1))
sig_mid = np.pad([1], (10, len(t)-11)) # 90 Hz
sig_far = np.pad([1], (100, len(t)-101))

# Lowered sampling frequency at basebanded signals
fft_close = scipy.fft.fft(sig_close, n=len(t))
fft_mid = scipy.fft.fft(sig_mid, n=len(t))
fft_far = scipy.fft.fft(sig_far, n=len(t))
freq_axis = scipy.fft.fftfreq(n=len(t), d=1/fs)

plt.figure(figsize=(8, 6))

# Plot each component
plt.subplot(3, 2, 1)
plt.plot(t, sig_close)
plt.title('Close Impulse')
plt.xlabel('Time [s]')
plt.ylabel('Amplitude')
plt.subplot(3, 2, 3)
plt.plot(t, sig_mid)
plt.title('Mid-Range Impulse')
plt.xlabel('Time [s]')
plt.ylabel('Amplitude')
plt.subplot(3, 2, 5)
plt.plot(t, sig_far)
plt.title('Far-Range Impulse')
plt.xlabel('Time [s]')
plt.ylabel('Amplitude')

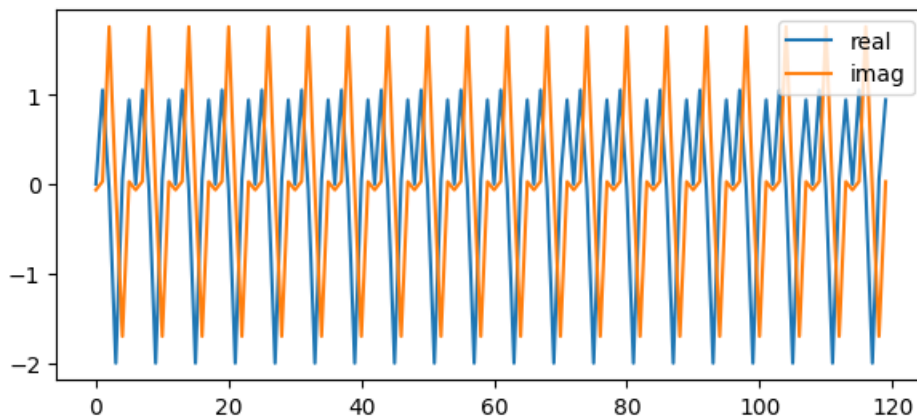
plt.subplot(3, 2, 2)
plt.plot(scipy.fft.fftfreq(n=len(t), d=1/fs), scipy.fft.fftshift(np.abs(fft_close)))
plt.plot(scipy.fft.fftfreq(n=len(t), d=1/fs), scipy.fft.fftshift(np.angle(fft_close)))
plt.subplot(3, 2, 4)
plt.plot(scipy.fft.fftfreq(n=len(t), d=1/fs), scipy.fft.fftshift(np.abs(fft_mid)))
plt.plot(scipy.fft.fftfreq(n=len(t), d=1/fs), scipy.fft.fftshift(np.angle(fft_mid)))
plt.subplot(3, 2, 6)
plt.plot(scipy.fft.fftfreq(n=len(t), d=1/fs), scipy.fft.fftshift(np.abs(fft_far)))
plt.plot(scipy.fft.fftfreq(n=len(t), d=1/fs), scipy.fft.fftshift(np.angle(fft_far)))
plt.xlabel("Frequency")
plt.tight_layout();
```

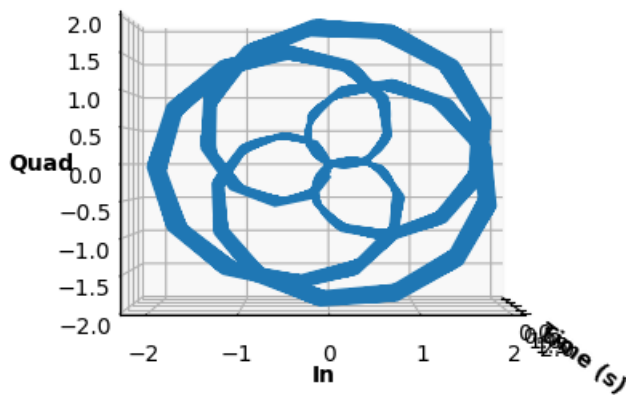


In [6]: *# Short example: Hilbert Transform to get our "analytic signal"*

```
sig_data = sine_cos(t)
sig_data = scipy.signal.hilbert(sig_data)
plt.figure(figsize=(7, 3))
plt.plot(sig_data.real, label="real")
plt.plot(sig_data.imag, label="imag")
plt.legend();

# Create figure and 3D axis
high_res_t = np.linspace(0, t_end, 500*t_end)
high_res_sig = scipy.signal.hilbert(sine_cos(high_res_t))
fig = plt.figure()
ax = fig.add_subplot(projection='3d')
ax.plot(high_res_t, high_res_sig.real, high_res_sig.imag)
ax.set_xlabel('Time (s)', fontweight='bold')
ax.set_ylabel('In', fontweight='bold')
ax.set_zlabel('Quad', fontweight='bold')
# ax.set_xlim(0, 0.25)
ax.view_init(elev=0, azim=-0);
```





Fourier Analysis

In the frequency domain, it can be easier to do **filtering, downsampling, frequency shifting, and time delaying**. We are only showing the downsampling and frequency shifting in this section.

- Citation for data in `manatee.npy` : Rycyk, A., V. Cargille, D. Bolaji, C. Factheu, U. Ejimadu, C. Berchem, and A. Takoukam Kamla. 2025. *Bioacoustic Dataset of African and Florida Manatee Vocalizations for Machine Learning Applications, 2020-2022 ver 2. Environmental Data Initiative*. <https://doi.org/10.6073/pasta/c73edcb4a36ed07aebfbe238a31ceb19>.
- The original sampling frequency was 44.1kHz.

```
In [7]: og_samples = np.load("manatee.npy")
fs = 44100
t_axis = np.arange(0, len(og_samples), step=1) / fs

# Normalize the waveform to range [-1, 1]
og_samples = (og_samples - og_samples.mean()) / (np.ptp(og_samples) / 2) # Take out the DC

n_fft = len(og_samples)
audio_fft = scipy.fft.fft(og_samples, n=n_fft)
freq_axis = scipy.fft.fftfreq(n_fft, d=1/fs)

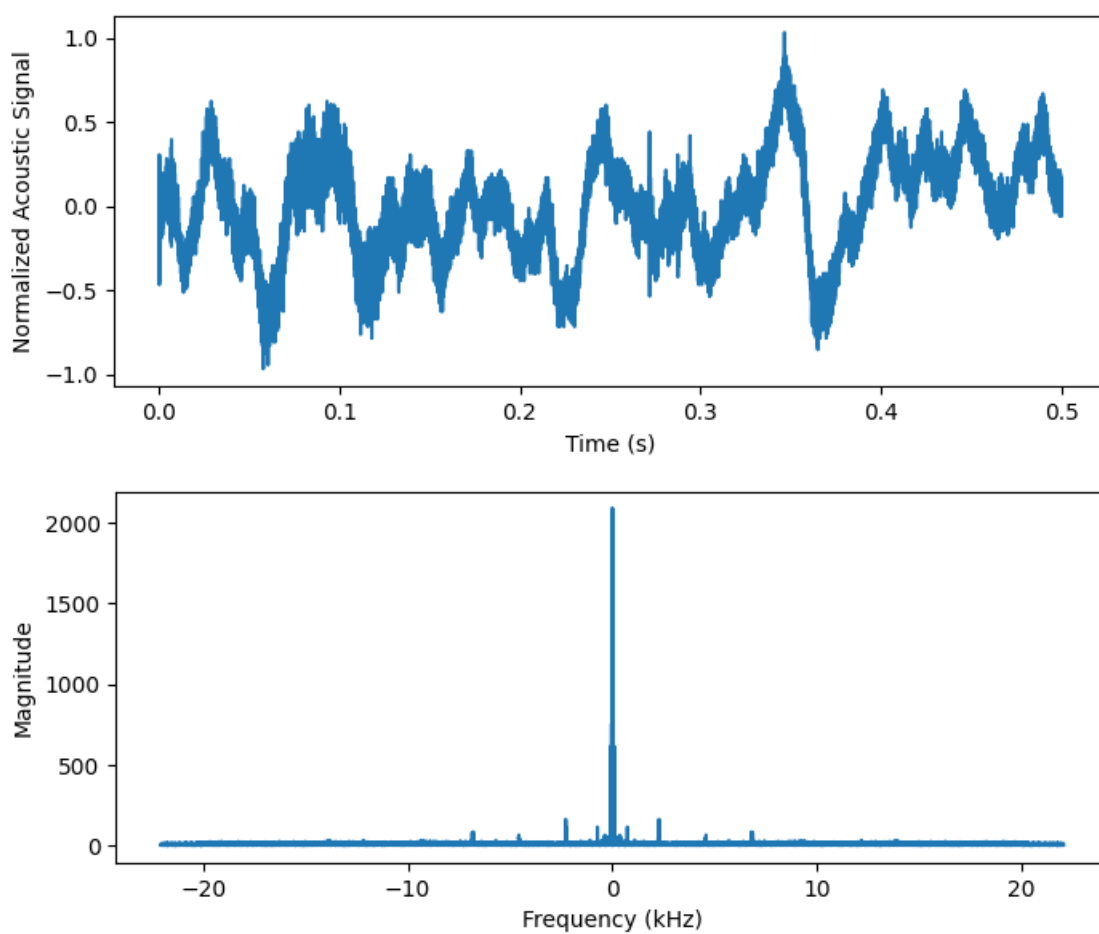
# Print summary
print(f"Audio duration: {len(og_samples)/fs:.2f} seconds")
print(f"Sample rate: {fs} Hz ({len(og_samples)} samples)")

plt.figure(figsize=(8, 3))
plt.plot(t_axis, og_samples.real)
plt.xlabel("Time (s)")
plt.ylabel("Normalized Acoustic Signal");

plt.figure(figsize=(8, 3))
plt.plot(scipy.fft.fftshift(freq_axis) / 1000,
         np.abs(scipy.fft.fftshift(audio_fft)))
plt.xlabel("Frequency (kHz)")
plt.ylabel("Magnitude");
```

Audio duration: 0.50 seconds

Sample rate: 44100 Hz (22051 samples)



Downsampling in the frequency domain

To upsample, you could simply pad with zeros at the center of the Fourier transform array (not the `fftshift` version).

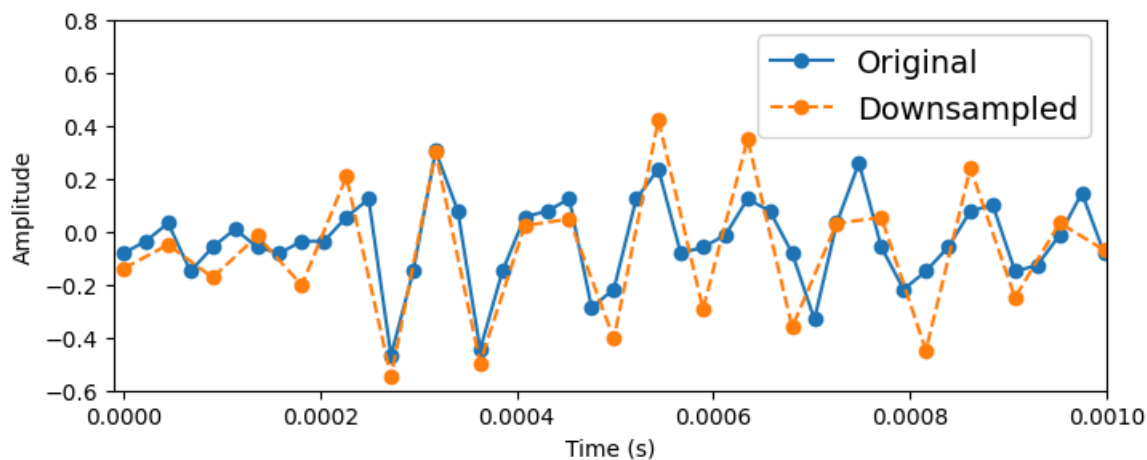
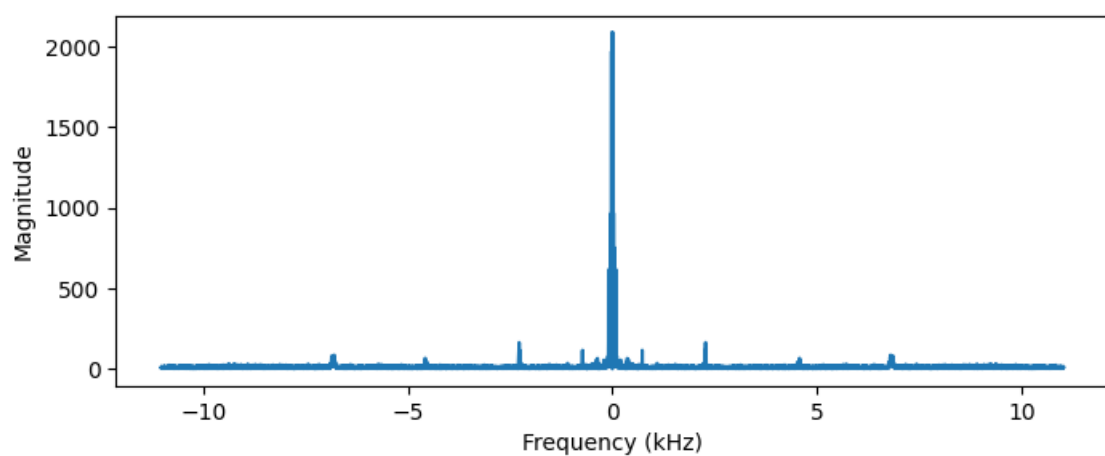
```
In [8]: new_fs = fs // 2
new_nfft = n_nfft // 2 # 22050 Hz

# Apply downsampling in frequency domain
ds_fft = np.concatenate((audio_fft[:new_nfft//2], audio_fft[-new_nfft//2:]))
new_freq_axis = scipy.fft.fftfreq(new_nfft, d=1/new_fs)

# Get the downsampled audio signal
ds_audio = scipy.fft.ifft(ds_fft)
new_t_axis = np.arange(0, len(ds_audio), step=1) / new_fs

plt.figure(figsize=(8,3))
plt.plot(scipy.fft.fftshift(new_freq_axis) / 1000,
         np.abs(scipy.fft.fftshift(ds_fft)))
plt.xlabel("Frequency (kHz)")
plt.ylabel("Magnitude")

plt.figure(figsize=(8,3))
plt.plot(t_axis, og_samples.real, "o-", label="Original")
plt.plot(new_t_axis, ds_audio.real, "o-", label="Downsampled")
plt.xlabel("Time (s)")
plt.ylabel("Amplitude")
plt.legend(fontsize=14)
plt.xlim(-1e-5, 0.001)
plt.ylim(-0.6, 0.8);
```



Frequency shifting in the frequency domain

This is the same as that shown above in the previous section "Signal Characteristics." Notice that we get some artifacts when shifting up (and aren't careful with how values get rolled around the array).

```
In [9]: f_shift = 1500
bin_spacing = new_fs / len(freq_axis)
bins_to_shift = int(round(f_shift / bin_spacing))
shifted_fft = np.roll(ds_fft, bins_to_shift)
dwnshift_audio = scipy.fft.ifft(shifted_fft)

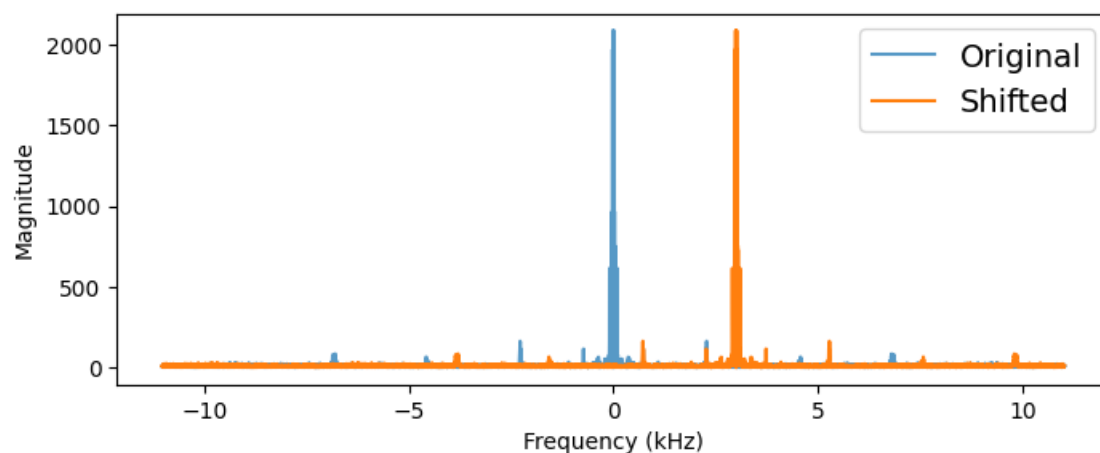
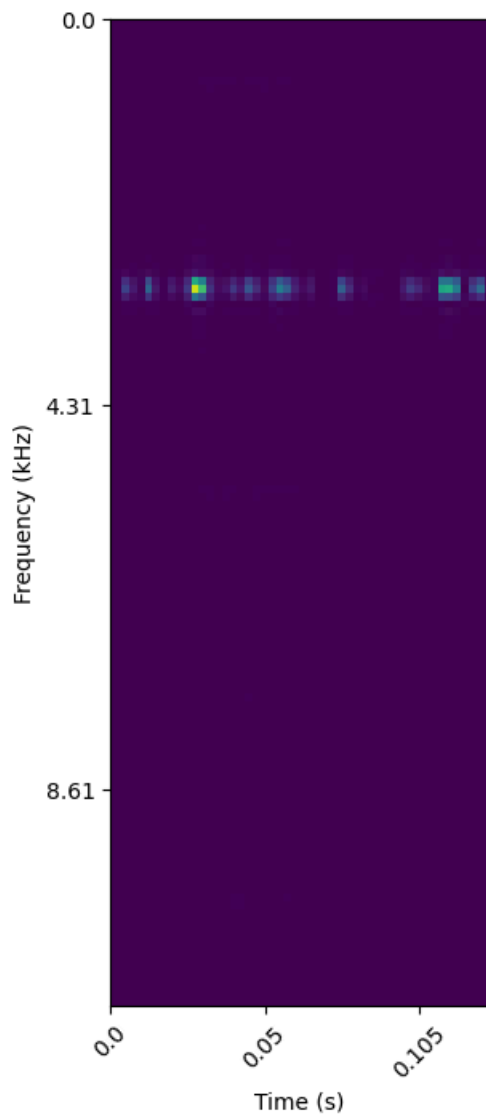
# Create visualization (spectrogram)
shift_f_spec, shift_t_spec, shift_audio_spec = scipy.signal.spectrogram(dwnshift_audio, fs=new_fs, nperseg=128, nfft=256)
shift_f_spec = np.round(shift_f_spec/1000, decimals=2)
shift_t_spec = np.round(shift_t_spec, decimals=2)

plt.figure(figsize=(4, 8))
plt.imshow(shift_audio_spec)
ax = plt.gca()
x_idx = np.asarray(ax.get_xticks(), dtype=np.uint)[1:-1]
y_idx = np.asarray(ax.get_yticks(), dtype=np.uint)[1:-1]

plt.xticks(x_idx, shift_t_spec[x_idx] / 2, rotation=45)
plt.yticks(y_idx, shift_f_spec[y_idx])
plt.xlabel("Time (s)")
plt.ylabel("Frequency (kHz)")
plt.xlim(0, len(shift_t_spec)//2);
plt.ylim(len(shift_f_spec)//2, 0);

# Display overall frequency domain
plt.figure(figsize=(8,3))
plt.plot(scipy.fft.fftfreq(new_freq_axis) / 1000,
         np.abs(scipy.fft.fftfreq(new_freq_axis) / 1000,
               np.abs(scipy.fft.fftshift(ds_fft))), alpha=0.75, label="Original")
plt.plot(scipy.fft.fftfreq(new_freq_axis) / 1000,
         np.abs(scipy.fft.fftshift(shifted_fft))), label="Shifted")
plt.xlabel("Frequency (kHz)")
plt.ylabel("Magnitude")
plt.legend(fontsize=14);
```

```
C:\Users\spencer.chang\AppData\Local\Temp\ipykernel_12740\1887606250.py:8: UserWarning: Input data is complex, switching to re
turn_onesided=False
shift_f_spec, shift_t_spec, shift_audio_spec = scipy.signal.spectrogram(dwnshift_audio, fs=new_fs, nperseg=128, nfft=256)
```



Matched Filtering

Filtering typically involves the removing of undesired parts of an input signal while preserving the desired parts of an input signal.

```
In [10]: recv_data = np.load("sas_recv_pings.npy") # Only taking one point along-track
proj_signal = np.load("sas_proj_signal.npy")
```

```
In [11]: compressed_data = np.load("sas_pulse_compress.npy")
```

```
In [12]: plt.imshow(np.abs(compressed_data), extent=(0, 5000, 0, 2000))
plt.axis('off');
```



```
In [13]: # Files to use: "sas_recv_pings.npy" and "sas_proj_signal.npy"
fs = 240e3
# TODO - Write code to perform pulse compression across the time-axis for every returned ping
time_ax = np.arange(recv_data.shape[1])/fs
freq_ax = scipy.fft.fftfreq(recv_data.shape[1], 1/fs)

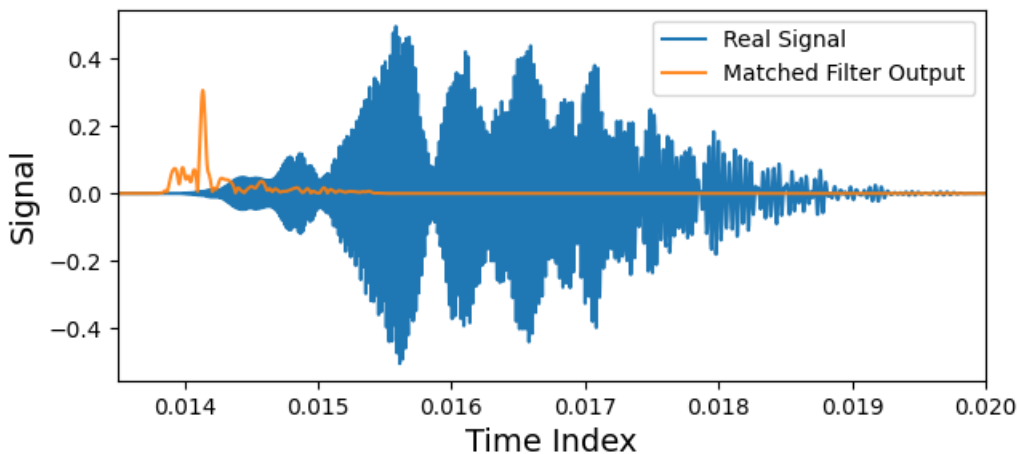
n_fft = scipy.fft.next_fast_len(max(recv_data.shape[-1], proj_signal.shape[0]))
recv_fft = scipy.fft.fft(recv_data, n=n_fft)
proj_fft = scipy.fft.fft(proj_signal, n=n_fft)

# Compute pulse compression on the received data.
proj_pow = (np.abs(proj_fft)**2).sum()
p_compress_fft = (recv_fft * proj_fft.conj().T) / proj_pow
p_compress_data = scipy.fft.ifft(p_compress_fft)
```

We can take one point along our synthetic aperture track to show how pulse compression helps with removing ambient noise.

```
In [14]: single_recv = recv_data[250]
single_compress = p_compress_data[250]

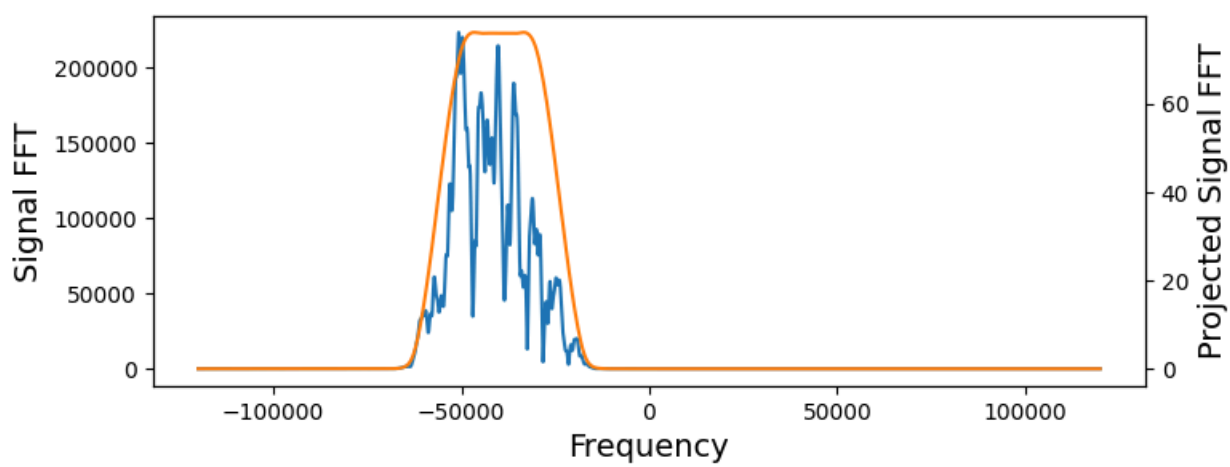
plt.figure(figsize=(7,3))
plt.plot(time_ax, single_recv.real / np.ptp(single_recv.real), label="Real Signal")
plt.xlabel("Time Index", fontsize=14)
plt.ylabel("Signal", fontsize=14)
plt.plot(time_ax, np.abs(single_compress), c="tab:orange", alpha=0.9, label="Matched Filter Output")
plt.xlim(0.0135, 0.02);
plt.legend();
```



```
In [15]: single_recv_fft = scipy.fft.fftshift(recv_fft[250])
single_compress_fft = scipy.fft.fftshift(p_compress_fft[250])
single_proj_fft = scipy.fft.fftshift(proj_fft)

plt.figure(figsize=(8,3))
plt.plot(scipy.fft.fftshift(freq_ax), np.abs(single_recv_fft), label="Real Signal")
plt.xlabel("Frequency", fontsize=14)
plt.ylabel("Signal FFT", fontsize=14)
ax = plt.twinx()
ax.plot(scipy.fft.fftshift(freq_ax), np.abs(single_proj_fft), c="tab:orange")
plt.ylabel("Projected Signal FFT", fontsize=14)
```

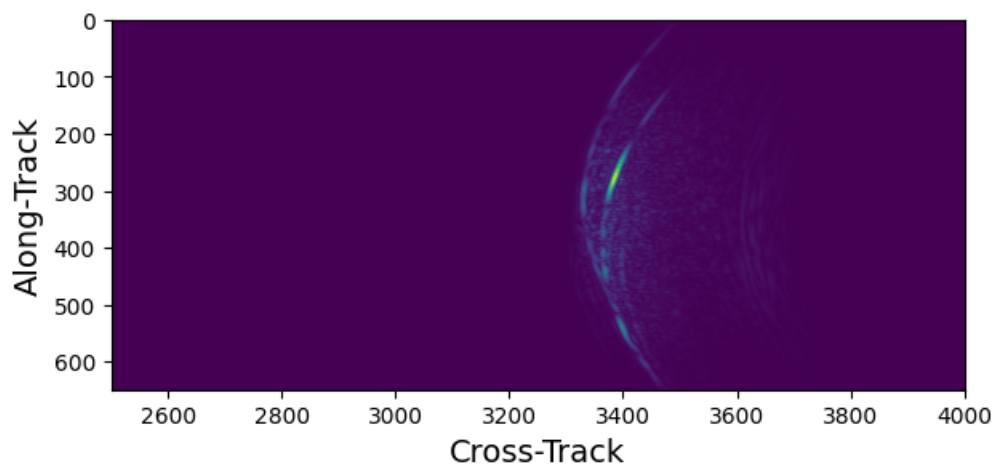
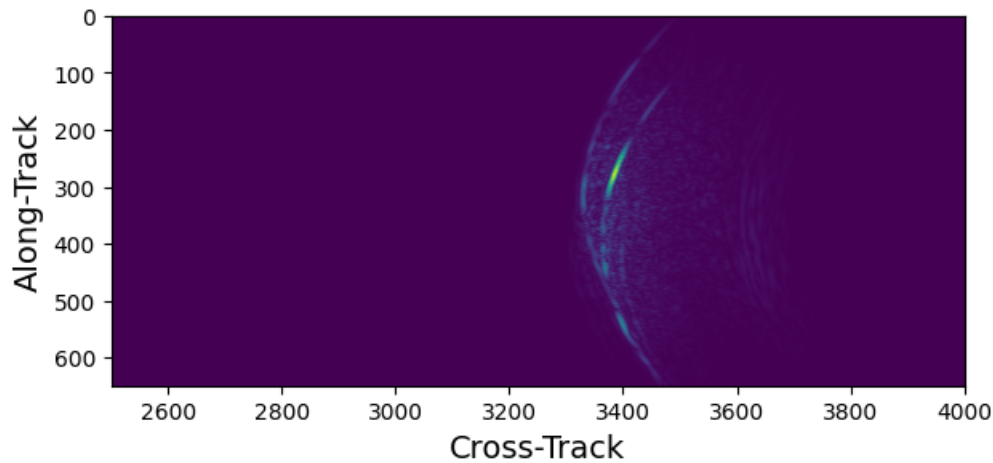
```
Out[15]: Text(0, 0.5, 'Projected Signal FFT')
```



When we plot the full, pulse-compressed synthetic aperture sonar (SAS) data, we get our "SAS smiles."

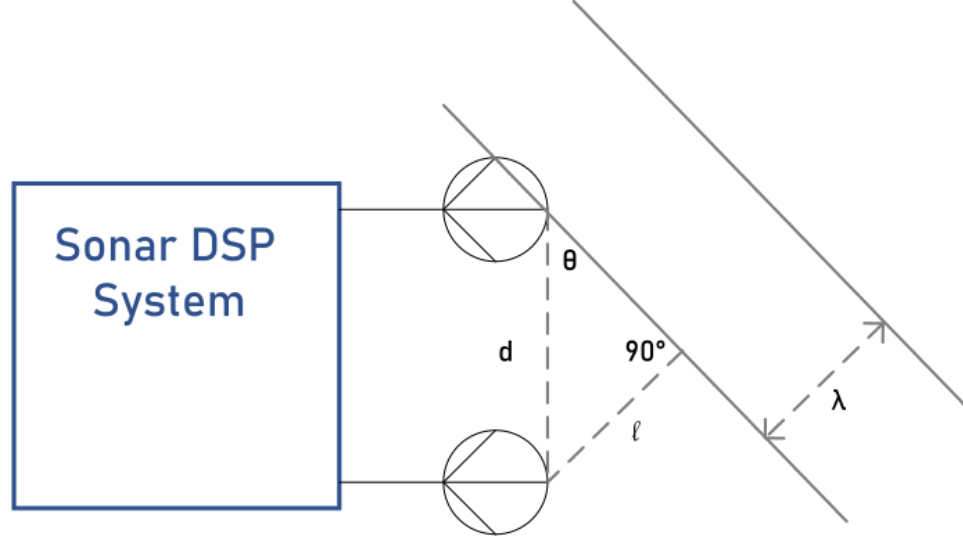
```
In [16]: plt.figure(figsize=(7,3))
plt.imshow(np.abs(p_compress_data))
plt.xlabel("Cross-Track", fontsize=14)
plt.ylabel("Along-Track", fontsize=14)
plt.xlim(2500, 4000);

plt.figure(figsize=(7,3))
plt.imshow(np.abs(compressed_data))
plt.xlabel("Cross-Track", fontsize=14)
plt.ylabel("Along-Track", fontsize=14)
plt.xlim(2500, 4000);
```



Delay-and-Sum Beamforming: Linear Receiver Array

Let's consider what happens when a sound wave hits a two-element receiver array. Each of the sound wave's peaks are represented by the lines below, so we will call this a "plane wave" going forward (ie. a "plane" of sound hits each receiver).



What we need to solve for is the time delay for the plane wave hitting the second (bottom) receiver. Notice that the distance from the initial plane wave to the second receiver is denoted by ℓ . Allowing transmission loss between elements to be negligible, we will define our time series equations for the receivers:

$$\begin{aligned} r_0(t) &= Ae^{j\omega t} \\ r_1(t) &= Ae^{j\omega(t-\tau)} \end{aligned}$$

How will we solve for τ ? The answer is that we'll use $\ell = c\tau$ to help us figure out the answer. Recall that c is the speed of sound underwater and could change slightly depending on how deep the receiver is.

We'll skip τ 's mostly straightforward derivation, but to see the details beforehand, feel free to expand the details below.

► Mathematical Derivation

Ultimately, if we expand the notation and break apart the complex exponentials for $r_1(t)$ above, we get

$$r_1(t) = Ae^{j\omega t} e^{-j2\pi f \frac{d}{c} \sin \theta} = Ae^{j\omega t} e^{-j2\pi \frac{d}{\lambda} \sin \theta}.$$

What happens when we include multiple receivers in the array and ensure that all have equal inter-element spacing? We can see that the receiver equations will be as follows:

$$\begin{aligned} r_0(t) &= Ae^{j\omega t} \\ r_1(t) &= Ae^{j\omega t} e^{-j2\pi \frac{d}{\lambda} \sin \theta} \\ r_2(t) &= Ae^{j\omega t} e^{-j2\pi \frac{2d}{\lambda} \sin \theta} \\ &\vdots \\ r_{N-1}(t) &= Ae^{j\omega t} e^{-j2\pi \frac{(N-1)d}{\lambda} \sin \theta} \end{aligned}$$

We can put this into vector form: $\mathbf{y} = [r_0(t_0) \ r_1(t_0) \ \dots \ r_{N-1}(t_0)]$. This way,

$$\begin{aligned} \mathbf{y} &= \hat{A} [1 \ e^{-jk_\theta} \ \dots \ e^{-j2\pi(N-1)k_\theta}] \\ &= \hat{A} \mathbf{a}_s(\theta) \end{aligned}$$

where $k_\theta = 2\pi \frac{d}{\lambda} \sin \theta$. We will call $\mathbf{a}_s(\theta)$ the "steering vector" pointed toward the angle θ .

```
In [17]: def w_mvdr(theta, X, d_cumsum):
    """
    Given some angle theta and signals X,
    compute the MVDR weight vector in that direction.
    Assuming that X's row shape determines the number of receivers
    """
    n_recv = X.shape[0]

    # Expecting theta in radians
    s_arr = np.exp(-2j * np.pi * d_cumsum[np.arange(n_recv)] * np.sin(theta))[:, np.newaxis]
    R_x = (X@X.conj().T) / X.shape[1] # X^H is Hermitian (complex) transpose
    R_inv = np.linalg.pinv(R_x)

    w = (R_inv@s_arr) / (s_arr.conj().T@R_inv@s_arr)
    return w
```

```
In [18]: def est_music(theta, X, d_cumsum, En=None, num_signals=1):
    """
    Computes the MUSIC pseudo-spectrum weight for a given angle theta.

    Args:
        theta (float): Angle of arrival in radians.
        X (np.ndarray): Sonar array data, shape (N_elements, M_samples).
        d_cumsum (np.ndarray): Element-wise distances from the first element
            divided by the wavelength, shape (N_elements,).
        En (np.ndarray): list of eigenvector arrays for the signal X. If None,
            calculate the list. 0/w, use En.
        num_signals (int): Expected number of noise signals (default is 1).

    Returns:
        float: The MUSIC pseudo-spectrum value for the given angle.
    """
    M = X.shape[1]

    eigenvalues = None
    if En is None:
        R = (X @ X.conj().T) / M

        eigenvalues, eigenvectors = np.linalg.eigh(R)

        # Sort eigenvalues and eigenvectors in descending order
        idx = np.argsort(np.abs(eigenvalues))[:,::-1]
        eigenvectors = eigenvectors[:, idx]
        eigenvalues = eigenvalues[idx]
        En = eigenvectors[:, num_signals:]

    a_theta = np.exp(-2j * np.pi * d_cumsum * np.sin(theta))
    a_theta = a_theta.reshape(-1, 1) # Format as column vector

    # 5. Compute the MUSIC pseudo-spectrum weight
    # Weight = 1 / (a^H * En * En^H * a)
    projection = a_theta.conj().T @ En @ En.conj().T @ a_theta

    # The result of the projection is theoretically real; extract the real part of the scalar
    music_pow = np.abs(1.0 / (projection.squeeze() + 1e-12))

    return music_pow, eigenvalues
```

```
In [19]: Nr = 5 # 5 elements
d = 0.5 # inter-element distance div'd by wavelength
d_dist = d * np.arange(Nr)

sample_rate = 1e6 # 1 MHz sampling
N = 10000 # number of samples to simulate
t = np.arange(N) / sample_rate # time vector

# Create tones to act as the transmitter signal
theta1 = 20 / 180 * np.pi # convert to radians
theta2 = 40 / 180 * np.pi
theta3 = -20 / 180 * np.pi

s1 = np.exp(-2j * np.pi * d_dist * np.sin(theta1)).reshape(-1,1) # Nr x 1
s2 = np.exp(-2j * np.pi * d_dist * np.sin(theta2)).reshape(-1,1)
s3 = np.exp(-2j * np.pi * d_dist * np.sin(theta3)).reshape(-1,1)

# we'll use 3 different frequencies. 1xN
tone1 = scipy.signal.hilbert(np.cos(2*np.pi*0.01e6*t)).reshape(1,-1)
tone2 = scipy.signal.hilbert(np.cos(2*np.pi*0.02e6*t)).reshape(1,-1)
tone3 = scipy.signal.hilbert(np.cos(2*np.pi*0.03e6*t)).reshape(1,-1)

X = s1 @ tone1 + 2*s2 @ tone2 + 0.1 * s3 @ tone3 # note the last one is 1/10th the power
n = scipy.signal.hilbert(np.random.randn(Nr, N)) #+ 1j*np.random.randn(Nr, N)
X = X + 0.1*n # 8xN
```

```
In [20]: theta_scan = np.linspace(-0.5*np.pi, 0.5*np.pi, 500) # 500 different thetas between -90 and +90 degrees
results = []
results_das = []
results_music = []
n_exp_sigs = 3

# Pre-calc eigen num's for MUSIC
R = (X @ X.conj().T) / X.shape[-1]
```

```

e_vals, e_vectors = np.linalg.eigh(R)
idx = np.argsort(np.abs(e_vals))[:, :-1]
e_vectors = e_vectors[:, idx]
e_vals = e_vals[idx]
En = e_vectors[:, n_exp_sigs:]

for theta_i in theta_scan:
    w_das = np.exp(-2j * np.pi * d_dist * np.sin(theta_i)) # Conventional, aka delay-and-sum, beamformer
    w_mv = w_mvdr(theta_i, X, d_cumsum=d_dist) # 3x1
    X_weighted = w_mv.conj().T @ X # apply weights
    X_das = w_das.conj().T @ X
    var_music, _ = est_music(theta_i, X, d * np.arange(X.shape[0]), En=En, num_signals=n_exp_sigs)

    power_dB = 10*np.log10(np.var(X_weighted)) # power in signal, in dB so its easier to see small and large Lobes at the same
    power_das_dB = 10*np.log10(np.var(X_das)) # power in signal, in dB so its easier to see small and large Lobes at the same
    music_out = 10*np.log10(var_music)

    results.append(power_dB)
    results_das.append(power_das_dB)
    results_music.append(music_out)

results -= np.max(results) # normalize
results_music -= np.max(results_music) # normalize
results_das -= np.max(results_das) # normalize

```

DAS vs MVDR

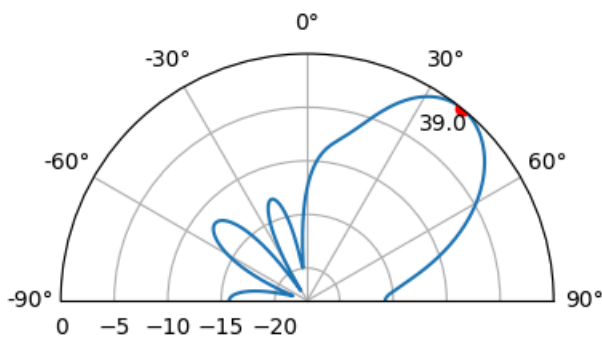
```

In [21]: ##### DELAY-AND-SUM BEAMFORMING PLOT ###
theta_max = theta_scan[np.argmax(results_das)]
# Plot all results
fig, ax = plt.subplots(1, 2, subplot_kw={'projection': 'polar'}, figsize=(10, 4))
ax[0].set_title("DAS Beamforming")
ax[0].plot(theta_scan, results_das) # MAKE SURE TO USE RADIAN FOR POLAR
ax[0].plot([theta_max], [np.max(results_das)], 'ro')
ax[0].text(theta_max - 0.1, np.max(results_das) - 4, np.round(theta_max * 180 / np.pi))
ax[0].set_theta_zero_location('N') # make 0 degrees point up
ax[0].set_theta_direction(-1) # increase clockwise
ax[0].set_rlabel_position(55) # Move grid labels away from other labels
ax[0].set_thetamin(-90) # only show top half
ax[0].set_thetamax(90)

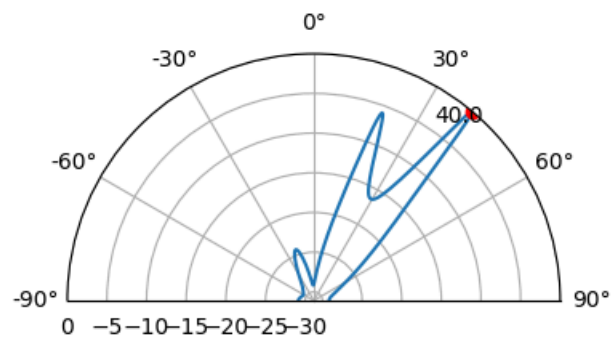
##### MINIMUM VARIANCE DISTORTIONLESS RESPONSE PLOT ###
theta_max = theta_scan[np.argmax(results)]
ax[1].set_title("MVDR Beamforming")
ax[1].plot(theta_scan, results) # MAKE SURE TO USE RADIAN FOR POLAR
ax[1].plot([theta_max], [np.max(results)], 'ro')
ax[1].text(theta_max - 0.1, np.max(results) - 4, np.round(theta_max * 180 / np.pi))
ax[1].set_theta_zero_location('N') # make 0 degrees point up
ax[1].set_theta_direction(-1) # increase clockwise
ax[1].set_rlabel_position(55) # Move grid labels away from other labels
ax[1].set_thetamin(-90) # only show top half
ax[1].set_thetamax(90)
plt.show()

```

DAS Beamforming



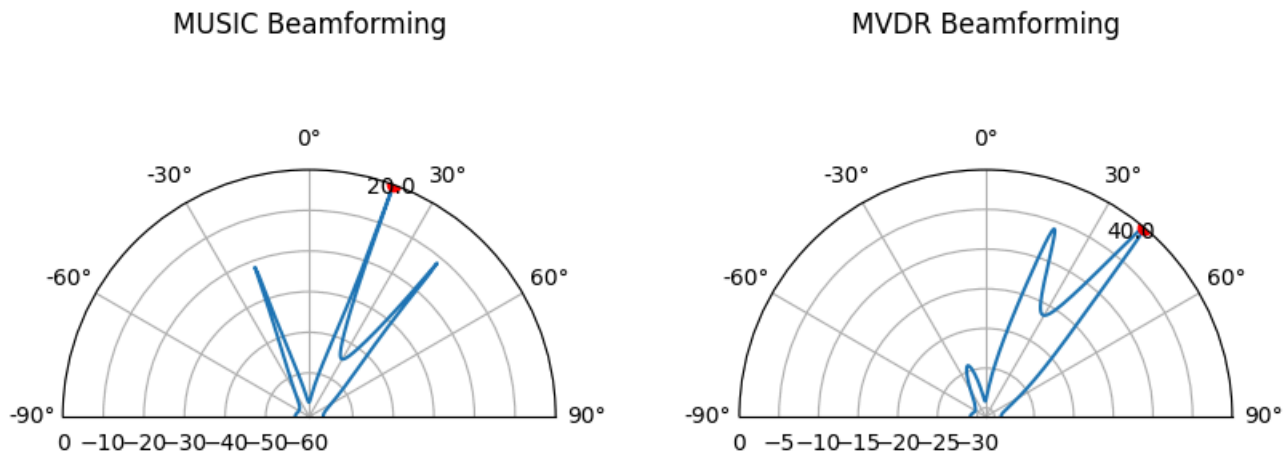
MVDR Beamforming



MVDR vs MUSIC

```
In [22]: ##### Multiple Signal Classification BEAMFORMING PLOT ###
theta_max = theta_scan[np.argmax(results_music)]
# Plot all results
fig, ax = plt.subplots(1, 2, subplot_kw={'projection': 'polar'}, figsize=(10, 4))
ax[0].set_title("MUSIC Beamforming")
ax[0].plot(theta_scan, results_music) # MAKE SURE TO USE RADIAN FOR POLAR
ax[0].plot([theta_max], [np.max(results_music)], 'ro')
ax[0].text(theta_max - 0.1, np.max(results_music) - 4, np.round(theta_max * 180 / np.pi))
ax[0].set_theta_zero_location('N') # make 0 degrees point up
ax[0].set_theta_direction(-1) # increase clockwise
ax[0].set_rlabel_position(55) # Move grid labels away from other Labels
ax[0].set_thetamin(-90) # only show top half
ax[0].set_thetamax(90)

##### MINIMUM VARIANCE DISTORTIONLESS RESPONSE PLOT ###
theta_max = theta_scan[np.argmax(results)]
ax[1].set_title("MVDR Beamforming")
ax[1].plot(theta_scan, results) # MAKE SURE TO USE RADIAN FOR POLAR
ax[1].plot([theta_max], [np.max(results)], 'ro')
ax[1].text(theta_max - 0.1, np.max(results) - 4, np.round(theta_max * 180 / np.pi))
ax[1].set_theta_zero_location('N') # make 0 degrees point up
ax[1].set_theta_direction(-1) # increase clockwise
ax[1].set_rlabel_position(55) # Move grid labels away from other Labels
ax[1].set_thetamin(-90) # only show top half
ax[1].set_thetamax(90)
plt.show()
```



Synthetic Aperture Sonar Beamforming

The basic math synthetic aperture sonar (SAS) beamforming imagery is straightforward. However, the most difficult part of the process is the bookkeeping. In other words, a programmer must keep all the variables in order and computed correctly for an image to be processed into a recognizable image (i.e., it looks like the seafloor, an object of interest).

Naïve Approach - TD-Backprojection

What we'll discuss briefly in this tutorial is the method of **backprojection**. The one-sentence explanation is that we compute the exact (expected) time-delay at *each* pixel in our ranging grid for *each* of our transducer elements at *each* of the platform's positions.

The five steps we can split this method into are:

1. Point Range
2. Time Delay
3. Invert Time Delay
4. Beamforming Summation
5. Beamformed Magnitude

In pseudocode, it goes something like the following:

1. Pulse compression of raw, complex signal data (using known projected signal).
2. If necessary, perform basebanding, filtering, and downsampling in the fast-time direction (cross-track).
3. **For each platform** position,....
 - A. **For each transducer** in the array,...
 - a. Compute the FFT of the transducer's time series.
 - b. **For each pixel** in the along-track and cross-track grid,...
 - i. Compute the time-delay
 - ii. Shift the transducer's frequency response by the time-delay amount across all frequencies.
 - iii. Add the mean across the transducer's frequency response.
 - iv. Store the mean in the pixel's variable.

Using [MASTODON](#) (PR, v0.22.6) to simulate ping data, let's make our own SAS image beamformer with TD-Backprojection.

```
In [23]: def pulse_compression(recv_signal:np.ndarray, pulse:np.ndarray):
        """
        Perform pulse compression on a given receiver's signal.

        recv_signal may be 2D; time series is expected on last axis (eg. -1)
        pulse is assumed to be 1D and a single projected signal.

        Edited with some code review by GenAI-mil.
        """
        # Assumption received signal always longer than pulse. Should probably be an error o/w
        n_sig = recv_signal.shape[-1]
        # fast_n = n_sig + pulse.shape[-1] - 1
        fast_n = n_sig
        fast_n = scipy.fft.next_fast_len(fast_n) # Suggested optimization
        pulse_E = np.sum(np.abs(pulse)**2)

        print(f"[Pulse Compression with {fast_n} samples...]")
        X_fft = scipy.fft.fft(recv_signal, n=fast_n, axis=-1)
        H_fft = scipy.fft.fft(pulse, n=fast_n)

        Y_fft = X_fft * H_fft.conj() / pulse_E
        return scipy.fft.ifft(Y_fft)[..., :n_sig]
```

```
In [24]: # Create a grid of ranging coordinates related to along-track and cross-track position.
def range_grid_calc(src_grid:np.ndarray, tgt_grid:np.ndarray, axis=-1):
    assert src_grid.shape == tgt_grid.shape, f"Improper shapes between src and tgt: {src_grid.shape, tgt_grid.shape}"
    return np.linalg.vector_norm(src_grid - tgt_grid, axis=axis, ord=2)
```

```
In [25]: # For each ping, compute the time delay and add the time-delayed series to a storage array the same size as the grid of ranging
# - Remember to apply the time delay across all possible frequencies for the ping's time series.
def process_single_ping(k, n_ele, stave_c, recv_pos, ele_loc, tgt_grid,
                       compressed_stave, freq_pts, arr_sum:int=-1,
                       verbose=0):
    recv_k = k * n_ele
    if verbose > 0:
        print_str = f"*** PLATFORM - {k} :: STAVE - {recv_k}"

    # Initialize blank grid for this ping
    ping_grid = np.zeros((tgt_grid.shape[0], tgt_grid.shape[1]), dtype=complex)
    if arr_sum < 0 or arr_sum > n_ele:
        arr_sum = n_ele # How many elements in the sas array to sum

    platform_pos = recv_pos[k:k+1, np.newaxis, :]
    for ele_id in range(arr_sum):
        ele_offs = np.repeat(platform_pos + ele_loc[:, ele_id],
                              repeats=tgt_grid.shape[1], axis=1)
        arr_ele_rep = np.repeat(ele_offs, repeats=tgt_grid.shape[0], axis=0)
        arr_ele_range = range_grid_calc(arr_ele_rep, tgt_grid)
        td = (2*arr_ele_range) / stave_c

        # Compute FFT of the ping data
        ping_fft = scipy.fft.fft(compressed_stave[recv_k+ele_id, :])

        if verbose > 0:
            print_str += f" :: ELE - {recv_k+ele_id}"

        # Vectorized phase shift per row
        for row in range(td.shape[0]):
```

```

        for col in range(td.shape[1]):
            # Need to re-apply phase shift for each point in range grid.
            freq_delays = np.exp(2j * np.pi * freq_pts * td[row, col])
            td_ping_fft = ping_fft * freq_delays
            ping_grid[row, col] += td_ping_fft.mean()

    if verbose > 0:
        print(print_str, flush=True)

    return ping_grid

def process_single_td_interp(k, n_ele, stave_c, recv_pos, ele_loc, tgt_grid,
                           compressed_stave, fs, arr_sum: int = -1):
    recv_k = k * n_ele
    if arr_sum < 0 or arr_sum > n_ele:
        arr_sum = n_ele

    n_rows, n_cols = tgt_grid.shape[:2]
    flat_tgt = tgt_grid.reshape(-1, 3)
    platform_pos = recv_pos[k]

    n_samples = compressed_stave.shape[1]
    time_axis = np.arange(n_samples) / fs

    ping_grid = np.zeros((n_rows, n_cols), dtype=complex)
    for ele_id in range(arr_sum):
        curr_ele_pos = platform_pos + ele_loc[:, ele_id]
        ranges = np.linalg.norm(flat_tgt - curr_ele_pos, axis=-1)
        td = (2.0 * ranges) / stave_c

        # 1D linear interpolation across fast-time replaces all frequency loops
        signal = compressed_stave[recv_k + ele_id, :] # (C,)
        ping_grid += np.interp(td, time_axis, signal, left=0.0, right=0.0).reshape(n_rows, n_cols)

    return ping_grid

```

```

In [26]: # Load data from the sonar pings
stave_data = StaveData("Pointlinear2.h5")

```

```

In [27]: # Some variables to make things easier later
do_bband = True
n_cpu_jobs = 8
use_n_ele = -1 # Set to -1 or > stave_data.n_ele to choose all possible elements in transducer array
batch_sz = 1
decimate_factor = stave_data.fs / stave_data.bw
print(decimate_factor)

print_stave_idx = 0 # Whether to print platform, stave, element indices when beamforming

```

4.8

```

In [28]: # Pulse Compression of input time series
use_n_ele = min(stave_data.n_ele, use_n_ele) if use_n_ele > 0 else stave_data.n_ele # Limit number of elements used
# Appears to both be in passband for pulse compression
compressed_stave = pulse_compression(stave_data.data, stave_data.signal.signal)

```

[Pulse Compression with 4800 samples...]

```

In [29]: # Baseband the data. One advantage is that computation is easier.
if do_bband:
    print("[Basebanding = yes]")

    bb_timesamp = np.arange(stave_data.n_time) / stave_data.fs
    bb_shift = np.exp(-2j * np.pi * stave_data.fc * bb_timesamp)[np.newaxis, :]
    compressed_stave = compressed_stave * bb_shift

    ### METHOD 1 - Lowpass to clean out shifted data, then decimate. ###
    butter_b, butter_a = scipy.signal.butter(3, stave_data.bw, btype='lowpass', fs=stave_data.fs)
    compressed_stave = scipy.signal.filtfilt(butter_b, butter_a, compressed_stave)

    # # Decimate now that things are basebanded
    # compressed_stave = scipy.signal.decimate(compressed_stave, decimate_factor)

    ### METHOD 2 - Fourier Domain; chop off 'zeroed' frequency components ###
    temp_f_ax = scipy.fft.fftfreq(stave_data.n_time, d=1/stave_data.fs)
    bw_idx = np.where(temp_f_ax == stave_data.bw/2)[0][0]

```

```

# print(bw_idx)
bb_fft = scipy.fft.fft(compressed_stave, axis=1)
bb_fft = np.hstack((bb_fft[:, :bw_idx], bb_fft[:, -bw_idx:]))
compressed_stave = scipy.fft.ifft(bb_fft)

print(f"[Signal Decimation by {decimate_factor} --> {compressed_stave.shape[-1]} time samples]")
dec_fs = stave_data.fs / decimate_factor
else:
    print("[Basebanding = no]")
    dec_fs = stave_data.fs

```

```

[Basebanding = yes]
[Signal Decimation by 4.8 --> 1000 time samples]

```

```

In [30]: r_ax = np.linspace(0, stave_data.r_max, compressed_stave.shape[-1]) # Range coord's
xr_ax = np.linspace(0, stave_data.dy * (stave_data.n_pings-1), stave_data.n_pings) # X-Range coord's (Edit: stave_data.recv.p
xr_ax = xr_ax - xr_ax.max()/2
z_motion = np.ones_like(xr_ax) * stave_data.motion[0]['z'] #np.asarray([v_motion['z'] for v_motion in stave_data.motion])

##### Get grid points for all "target locations" #####
stave_data.recv.positions = np.concatenate( # NOTE - X-Y swapped because bearing=-90-deg
    (stave_data.recv.positions[1:2, :],
     stave_data.recv.positions[0:1, :],
     stave_data.recv.positions[2:3, :]), axis=0
)
platform_loc = np.hstack((np.zeros_like(xr_ax)[: , np.newaxis], xr_ax[:, np.newaxis], z_motion[:, np.newaxis]))

#### SET 'TARGET' LOCATIONS ####
grid_xr = np.linspace(0, stave_data.dy * (stave_data.n_pings-1), stave_data.n_pings) # * use_n_ele
grid_xr = grid_xr - grid_xr.max()/2
r_pts, xr_pts = np.meshgrid(r_ax, grid_xr)
# Range, X-Range, Depth
tgt_loc_grid = np.concatenate((r_pts[:, :, np.newaxis],
                                xr_pts[:, :, np.newaxis],
                                10*np.ones_like(r_pts)[..., np.newaxis]), axis=-1)

## Compute each element location's specific range grid to get proportional change in time delay.
n_pts = stave_data.n_pings # * use_n_ele
total_n_pts = stave_data.n_pings # * stave_data.n_ele
td_bf_sum = np.zeros(tgt_loc_grid.shape[:-1], dtype=np.complex64)
f_axis = scipy.fft.fftfreq(compressed_stave.shape[-1], d=1/dec_fs) + stave_data.fc
print(f"[Freq. Range --> [{f_axis.min():e}, {f_axis.max():e}]\n")

print(f"***** TD-beamforming for {n_pts} of {total_n_pts} points ({use_n_ele} elements) *****\n")
results = Parallel(n_jobs=n_cpu_jobs, batch_size=batch_sz, verbose=10)(
    delayed(process_single_ping)(
        k,
        stave_data.n_ele,
        stave_data.c,
        platform_loc,
        stave_data.recv.positions,
        tgt_loc_grid,
        compressed_stave,
        f_axis,
        arr_sum=use_n_ele,
        verbose=print_stave_idx
    ) for k in range(0, total_n_pts) #, stave_data.n_ele
)
# Add up all responses...
td_bf_sum = np.abs(np.sum(results, axis=0))*2
td_bf_sum = td_bf_sum / (td_bf_sum.max() + 1e-12)

```

```

[Freq. Range --> [1.750000e+05, 2.249500e+05]

```

```

***** TD-beamforming for 25 of 25 points (13 elements) *****

```

```

[Parallel(n_jobs=8)]: Using backend LokyBackend with 8 concurrent workers.
[Parallel(n_jobs=8)]: Done 2 tasks | elapsed: 11.2s
[Parallel(n_jobs=8)]: Done 9 tasks | elapsed: 21.2s
[Parallel(n_jobs=8)]: Done 13 out of 25 | elapsed: 21.4s remaining: 19.8s
[Parallel(n_jobs=8)]: Done 16 out of 25 | elapsed: 21.8s remaining: 12.2s
[Parallel(n_jobs=8)]: Done 19 out of 25 | elapsed: 31.2s remaining: 9.8s
[Parallel(n_jobs=8)]: Done 22 out of 25 | elapsed: 31.4s remaining: 4.2s
[Parallel(n_jobs=8)]: Done 25 out of 25 | elapsed: 39.1s finished

```

```

In [32]: cmap_choice = "bone" # "pink", "bone"

# Crop the padded TD-beamforming array

```

```

print(f"***** Post-processing and Figure Plotting *****\n")

# Get maximum response; plot outputs
max_match = np.argmax(np.abs(compressed_stave).max(axis=0))
max_resp_idx = np.argmax(td_bf_sum.max(axis=0))
plt_r_axis = np.round(np.linspace(stave_data.r_min, stave_data.r_max,
                                num=compressed_stave.shape[-1]), decimals=1)
plt_trk_axis = np.round(xr_ax, decimals=1)

fig, axes = plt.subplots(4, 1, figsize=(10, 9), layout='constrained')
axes[0].imshow(np.abs(compressed_stave),
               extent=[plt_r_axis.min(), plt_r_axis.max(), plt_trk_axis.min(), plt_trk_axis.max()],
               cmap=cmap_choice)
axes[0].set_title("Max Matched Filter Response")
axes[0].set_xlabel("Cross-Track (m)")
axes[0].set_ylabel("Along-Track (m)")
ax_lim = max(0, max_match-150), min(len(plt_r_axis), max_match+150)
axes[0].set_xlim(plt_r_axis[ax_lim[0]], plt_r_axis[ax_lim[1]])

#### NOTE - The additional 1e-6 has an effect of increasing the lower bound of the displayed image.
#### It mainly helps us view the result more clearly.
axes[1].imshow(10 * np.log(td_bf_sum + 1e-6),
               extent=[plt_r_axis.min(), plt_r_axis.max(), plt_trk_axis.min(), plt_trk_axis.max()],
               cmap=cmap_choice)
axes[1].set_title("Beamforming at Max Response")
axes[1].set_xlabel("Cross-Track (m)")
axes[1].set_ylabel("Along-Track (m)")
ax_lim = max(0, max_match-150), min(len(plt_r_axis), max_match+150)
axes[1].set_xlim(plt_r_axis[ax_lim[0]], plt_r_axis[ax_lim[1]])

axes[2].plot(10 * np.log(td_bf_sum[:, max_resp_idx] + 1e-6))
axes[2].set_xticks(np.arange(td_bf_sum.shape[0])[::4], plt_trk_axis[::4])
axes[2].set_title(f"Max Response @ Cross-Track {plt_r_axis[max_resp_idx]:.2f}m")
axes[2].set_xlabel("Along-Track (m)")
axes[2].set_ylabel("Magnitude")

axes[3].plot(10 * np.log(td_bf_sum[td_bf_sum.shape[0]//2,...] + 1e-6))
axes[3].set_title("Cross-Track Result @ Center of Track")
axes[3].set_xticks(np.arange(td_bf_sum.shape[-1])[::50], plt_r_axis[::50])
axes[3].set_xlabel("Cross-Track (m)")
axes[3].set_ylabel("Magnitude");

```

***** Post-processing and Figure Plotting *****

